

TMC DAY 2 TASK

The SDLC — and How AI Is Rewriting It

Day: 2

Date: 2026-07-03

Intern Name: Abdullah Bin Zubair

University & Year: GIK Institute of Engineering Sciences and Technology, Junior (3rd year)

Team Name: TMC intern

AI Tools Used: Claude

Answers

1. SDLC stages, one sentence each:
 - Requirements — figure out what the software needs to do and for whom.
 - Design — plan architecture before coding.
 - Development — write the software.
 - Testing — check it works before users do.
 - Deployment — release to production.
 - Maintenance — fix and improve it post-launch.
2. Waterfall vs Agile:
 - Waterfall completes each stage fully before starting the next, in strict sequence.
 - Agile repeats short cycles (sprints) of build-test-adjust continuously instead of doing it all once.
3. How AI is changing each stage:
 - Requirements — drafts specs/user stories from conversations.
 - Design — proposes architecture options/diagrams.
 - Development — coding assistants (Copilot/Claude) write and complete code live.
 - Testing — auto-generates test cases including edge cases.
 - Deployment — reviews pull requests and flags risky changes before merge.
 - Maintenance — AIOps monitors production and predicts/flags failures automatically.
4. Risks of leaning on AI:
 - hallucinated code (calls to functions/libraries that don't exist), insecure code if unreviewed, over-trust leading developers to stop checking output, and the fact that AI doesn't understand real business intent — a human must still verify.
5. Agentic AI / AI-first development:
 - AI systems that autonomously plan, execute multi-step tasks, and act with minimal human supervision, rather than just responding to single prompts — spanning multiple SDLC stages (plan, code, test, explain) alongside human developers. This mirrors TMC's AiNow idea that AI should be woven into every layer of the business, not added as a single bolted-on tool.

Deliverable

SDLC × AI Table

Stage	Traditional	AI-Augmented	Risk to Watch
Requirements	Manual interviews/specs	AI drafts specs from conversations	Misses unstated business context
Design	Manual architecture diagrams	AI proposes architecture from description	Generic patterns that don't fit real constraints
Development	Manual coding	Coding assistants generate/complete code	Hallucinated or insecure code if unreviewed
Testing	Manual test case writing	AI auto-generates test cases	Misses business-specific edge cases
Deployment	Manual code review	AI reviews PRs, flags risky changes	Over-trusting AI "approval" without human sign-off
Maintenance	Manual log/alert monitoring	AIOps predicts/flags anomalies	False positives/negatives from miscalibration

"Where Software Is Heading"

In five years, I don't think the software engineer's job disappears — it moves up a level. Agentic AI is already handling full slices of the SDLC on its own: writing code, running tests, debugging, even opening pull requests without a human at every step. What doesn't change is judgment — deciding what "correct" means for a real business, weighing tradeoffs, and being accountable when something breaks in production. Engineers will spend less time typing code and more time specifying intent, reviewing AI output critically, and catching the subtle mistakes AI doesn't know it's making. That mirrors TMC's own AiNow philosophy: AI isn't replacing the professional, it's becoming the layer underneath everything they do. The job survives — it just moves from "writer of code" to "director and verifier of AI-written code."

Prompt Log

- 1". Explain each SDLC stage in one plain sentence, and give one concrete example of how AI is changing that stage today."
- 2". What is agentic AI / AI agents in software development — how is this different from a coding assistant like Copilot?"
- 3". What are the real risks and limits of relying on AI to write software — give specific failure modes, not generic warnings."
4. "Give me the strongest argument on both sides of: if AI can write code, what is a software engineer's job in 5 years?"

Sources & Verification

coderio.com/blog/innovation/agentic-ai-software-development-changes-tools-start-making-decisions/ — Forrester analyst's definition of agentic software development, cited in deliverable

firecrawl.dev/blog/agentic-ai-trends — definition of agentic AI as autonomous, tool-using, minimal-supervision systems

Reflection

Most surprising thing I learned: How far "agentic AI" already goes beyond a coding assistant — it's not just autocomplete anymore, it can plan, write, test, debug, and even open pull requests across multiple SDLC stages in one session with minimal human input at each step.

One thing I'm still unsure about: Where the line actually sits between an AI agent being "trusted to act" versus "needing sign-off" at each stage — the research made agentic AI sound further along in production use than I expected, and I'm not sure how much of that is already standard practice versus still early/experimental.

How my team helped: We compared our SDLC×AI tables stage by stage and disagreed most on the Deployment row — some of us had AI reviewing code as fully autonomous, others treated it as advisory only. We settled on "flags risky changes, human still approves" after checking how these review tools are actually described.

Time spent (approx): 2 hour 15 minutes